When Does Kernel Fusion Help?
A Measured Study of Batch-1 Edge Vision Pipelines on a Laptop GPU

Utkarsh Sengar
Indian Institute of Technology Dharwad
cs23bt026@iitdh.ac.in

Abstract

Custom GPU kernels for image preprocessing and detector postprocessing are often proposed to cut latency in single-stream (batch-1) vision pipelines. We implement two such kernels in Triton, a fused `uint8` HWC→normalized CHW preprocessor and a single-pass YOLO candidate filter with in-kernel stream compaction, and measure them in isolation and inside two complete pipelines (a 256×256 convolutional autoencoder and pretrained YOLOv8n at 640×640) on an RTX 4050 Laptop GPU. We separate *device time* (GPU execution with launch overhead hidden) from *host latency* (what the application waits for). In isolation, the preprocessing kernel is 5.60–19.19× faster than PyTorch eager on the GPU and 1.00–1.56× as fast as TorchInductor, but for small frames a single call's host latency is dominated by dispatch. The fused filter cuts host latency 1.62× relative to eager (218.8 → 135.2 µs), yet its kernel accounts for only 9.2 µs of that. End to end, swapping in either kernel changes pipeline latency by less than run-to-run variation, while capturing the static part of each pipeline in a CUDA graph reduces latency by 48.3% (anomaly) and 53.5% (detection). At batch 1 on this hardware, launch overhead, not memory traffic, decides the outcome. We also document three benchmarking pitfalls, each of which changed a conclusion during this study, and a compiler bug that silently corrupts compiled YOLOv8n outputs.

1 Introduction

Inspection and detection systems often process one camera frame at a time under a latency budget. Around the neural network, such pipelines run non-neural steps: layout conversion and normalization before the model, candidate filtering and non-maximum suppression (NMS) after it. Written as framework operator chains [1], these steps launch several small kernels and materialize intermediate tensors. Fusing them into custom kernels [2] is a common recommendation. This paper asks a narrow question: *on a batch-1 pipeline, how much latency does such fusion remove, compared with what the GPU compiler and CUDA graphs already provide?*

Contributions. (1) Two Triton kernels: a fused preprocessor that folds normalization into one FMA per element, and a single-pass candidate filter that reduces 80 class scores, thresholds, decodes boxes and compacts survivors on the GPU with one host synchronization. (2) A measurement protocol that reports GPU device time and host latency separately, with cold and warm caches and interleaved repeated blocks. (3) Results against PyTorch eager, TorchInductor [3] and manual CUDA-graph capture [4], from isolated kernels up to full pipelines, with Nsight Compute [5] counters. (4) Benchmarking pitfalls and a correctness bug: a hidden synchronization inside a timed region, which in an earlier version of this work produced a spurious 5× slowdown; cache flushes that hide launch overhead or charge dirty-line write-back to the measured kernel; and a TorchDynamo mis-trace that silently corrupts `torch.compile`d YOLOv8n outputs. All code, raw results and the generator of every number in this paper are public.[1]

2 Background

Operator fusion is a central technique of ML and image-processing compilers such as Halide [6], TVM [7] and TorchInductor [3], and of hand-written kernels such as FlashAttention [8]. It removes intermediate global-memory traffic and kernel launches. Both savings are bounded by Amdahl's law [9]: they only matter if the fused stages make up a meaningful part of end-to-end time. GPU memory behaviour (coalescing, sector use, caching) is analysed with microbenchmarks and hardware counters [5, 10]. CUDA graphs [4] remove per-kernel launch cost by replaying a captured sequence of kernels. Libraries such as DALI [11] and TensorRT [12] target the same pipeline stages in production.

3 Kernels

Kernel 1: preprocessing. The kernel computes $y_{c,h,w} = x_{h,w,c} \cdot s_c + b_c$ with $s_c = 1/(255\sigma_c)$ and $b_c = -\mu_c/\sigma_c$ folded on the host. A 1D grid assigns consecutive pixels to each program (block size autotuned). Each pixel needs three byte loads at stride 3 and three coalesced plane stores. Compulsory DRAM traffic is $15P$ bytes for P pixels in FP32. The 5-kernel eager chain (`to`, `div`, `sub`, `div`, `contiguous`) moves up to $111P$ bytes when no intermediate stays in cache. FP16 output is computed in FP32 and rounded once. A 2D-tiled variant performs the same arithmetic on 16×16-pixel tiles, so each tile row touches only 48 contiguous input bytes.

[1] https://github.com/sengarutk/cuda-kernels-edge-vision

Table 1: Isolated preprocessing, FP32 device time p50 (µs), cold L2.

Resolution	Eager	Inductor	Triton 1D	Triton 2D	GB/s
256×256	28.7	5.1	5.1	7.2	192.0
640×640	71.7	14.3	11.3	23.6	545.5
1280×720	210.9	28.7	18.4	48.0	750.0
1920×1080	1493.5	80.9	77.8	109.6	399.7
3840×2160	6199.3	887.8	596.0	618.5	208.8

Kernel 2: candidate filter. YOLOv8 [13, 14] produces a $[1, 84, N]$ tensor: 4 box values and 80 sigmoid class scores per candidate. One program handles a block of candidates. A compile-time-unrolled loop over classes keeps the running max and argmax in registers, with coalesced loads in this channels-first layout. Survivors (score ≥ 0.25) are decoded from (c_x, c_y, w, h) to corners and compacted in the kernel: each program reserves output slots with one atomic add and orders its survivors with a prefix sum (cf. the decoupled scan of Merrill and Garland [15]). The host reads the survivor count once. The eager baseline follows the Ultralytics pre-filter: one reduction, one `nonzero` (also a single synchronization), then argmax and box conversion on the survivors only.

4 Methodology

Setup. NVIDIA GeForce RTX 4050 Laptop GPU (sm_89, 20 SMs, 24.0 MB L2, 192 GB/s peak DRAM), driver 581.86, Ubuntu 24.04.4 LTS under WSL2, PyTorch 2.5.1+cu121, Triton 3.1.0, Nsight Compute 2025.3.1.

Timing. *Device time* uses CUDA events, with the stream prefilled by a spin kernel that outlasts the host's enqueue time, so the GPU never waits for the CPU inside the timed window. *Host latency* is wall-clock time from the call to a completed `synchronize`. For cold-L2 runs, a 64 MB buffer (more than twice the L2) is *read* before every call, outside the timed window. Reading matters: a write-based flush leaves the L2 full of dirty lines, and their write-back is then charged to the measured kernel (Section 6). Isolated kernels run 300 timed calls after 30 warm-up calls. Pipelines run 10 interleaved blocks of 300 frames per configuration, in an order that rotates per block; we report pooled percentiles and the spread of block medians.

Workloads. Preprocessing uses random frames from 256×256 to 3840×2160. The filter and the detection pipeline use real YOLOv8n outputs for the Ultralytics `bus.jpg` scene (8400 candidates, 46 above threshold, 5 detections after NMS at 640×640). The anomaly autoencoder has seeded random weights: we measure latency, not accuracy [16]. Every kernel output is verified against PyTorch before timing (max FP32 deviation 7.2e-07).

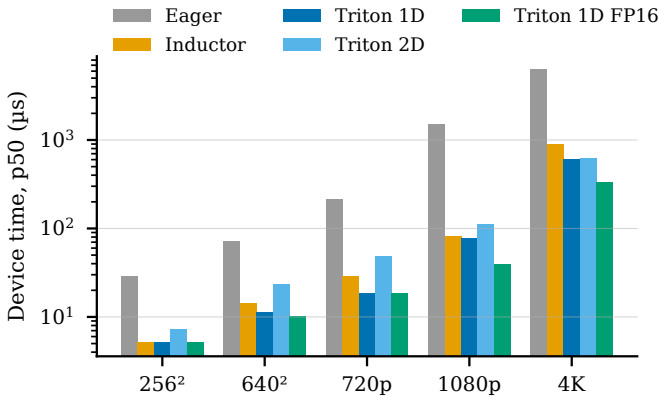


Figure 1: Isolated preprocessing device time (log scale), cold L2.

5 Results

5.1 Preprocessing in isolation

Table 1 and Figure 1 show device times with a cold L2. The fused kernel is 5.60–19.19× faster than the 5-kernel eager chain. TorchInductor fuses the same chain automatically. Relative to it, the Triton kernel is 1.00–1.56× as fast in FP32 (above 1 means faster) and 1.00–1.39× in FP16, so hand-written fusion still has a margin over the compiler on some shapes. That margin is measured in microseconds, though. Dispatching any kernel from Python costs tens of microseconds, so a single call's *host latency* improves over eager by 8.14× at 1080p, where GPU work dominates, but only 1.12× at 256×256. There the Triton call takes 110.4 µs on the host for 5.1 µs of GPU work.

5.2 2D tiling

With identical, synchronization-free dispatch, the 2D-tiled variant takes 1.04–2.60× the device time of the 1D kernel. Nsight Compute (Table 2, Figure 2) attributes this to memory access, not register pressure. At 1080p both kernels reach full theoretical occupancy, but the 2D kernel's global-load sector efficiency is 5.8% against 16.7%, and its dominant stall is mio throttle (41.3% of stalls), a queue for memory instructions. The 1D kernel is dominated by long scoreboard (51.6%), i.e. waiting on DRAM. Short 16-pixel rows issue more, smaller memory requests. An earlier version of this study reported a 5× slowdown caused by register pressure. That came from the 2D dispatcher reading CUDA-tensor normalization constants with `.tolist()`, a device synchronization inside the timed region that the 1D path did not perform.

5.3 Candidate filter

On the real 640×640 output (8400 candidates, 46 kept), the fused filter takes 135.2 µs of host latency against 218.8 µs for eager and 241.2 µs for Inductor (warm L2), a 1.62× reduction. At 1280×1280 (33600 candidates) the reduction is 1.55×. The kernel itself runs for 9.2 µs on the GPU. Launching it from Python without any synchronization still costs 58.2 µs of host

Table 2: Nsight Compute, largest kernel per target (base clocks, caches flushed). Occ.: achieved occupancy; Ld eff.: global-load sector efficiency.

Kernel	µs	Regs	Occ. %	DRAM %	Ld eff. %
Triton 1D, 1080p	89.6	20	82.8	84.0	16.7
Triton 2D, 1080p	159.8	28	94.6	62.5	5.8
Triton 1D, 640^2	13.3	38	74.7	47.2	8.3
Inductor, 640^2	15.2	27	84.1	41.4	8.4
Filter $[84, N]$	17.8	40	27.0	77.4	100.0
Filter $[N, 84]$	21.0	47	16.6	69.0	12.5

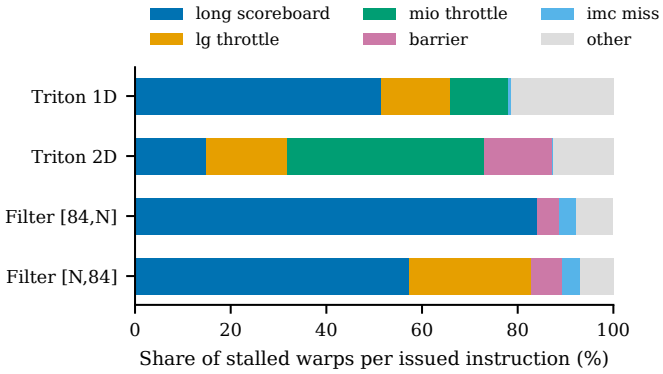


Figure 2: Warp-stall breakdown from Nsight Compute (top reasons; rest grouped as "other").

latency, so the filter's remaining cost is dispatch and the one unavoidable read-back, not GPU work. Reading YOLOv8's native channels-first layout matters: on the row-major $[N, 84]$ layout the same kernel is 1.33× slower (warm) because class loads are strided.

5.4 End-to-end pipelines

Figure 3 shows end-to-end latency. In the anomaly pipeline, replacing eager preprocessing with the Triton kernel changes p50 latency by -0.4% (positive means faster), within the 8.6–8.8% spread of block medians. In the detection pipeline, using both kernels changes it by 2.9%. Figure 4 explains why: YOLOv8n inference takes 5059.5 µs of host latency but only 3095.6 µs on the GPU. The pipeline is bound by launching the model's kernels, and preprocessing and filtering together are a small fraction of the frame. Compiling the model with TorchInductor reduces detection latency by 23.3%. Capturing H2D, preprocessing and the model in one CUDA graph reduces it by 50.4% with eager kernels and 53.5% with both Triton kernels (p50 2.893 ms). The anomaly pipeline shows the same pattern: a CUDA graph reduces latency by 47.1% (eager preprocessing) and 48.3% (Triton). With launches removed, the Triton configuration's median is 6.4% lower than the eager one for detection, where the fused filter runs outside the graph, and 2.4% lower for the anomaly pipeline. Both differences are still within the run-to-run spread of block medians (11.5% for the detection graph configuration), so we do not claim them as significant.

6 Discussion

Where fusion pays. The fused kernels remove most of the GPU work of their stages; at 1080p the preprocessing kernel sustains 84.0% of peak DRAM throughput under Nsight Compute. But at batch 1 on this system, the GPU time they save is microseconds, while each Python-level dispatch costs tens of microseconds and the model launches hundreds of kernels. Removing launches (CUDA graphs, model compilation) therefore dominates. A custom kernel is most useful where the compiler cannot reach it (data-dependent compaction, as in Kernel 2) or inside a graph-captured or C++ runtime, where its smaller GPU time is no longer hidden behind dispatch.

Measurement pitfalls. Three effects changed conclusions while this study was being done. First, any synchronization inside a timed region (here an innocuous `.tolist()`) turns kernel timing into host-latency timing for one side of a comparison only. Second, a cache-flush kernel placed before each timed call also hides launch overhead behind the flush, so "isolated" GPU times silently exclude dispatch. Third, the common write-based flush (`zero_()`) leaves the L2 full of dirty lines: the 1080p preprocessing kernel writes 8.6 MB to DRAM during its own execution under Nsight Compute, but under a write-based flush it must also evict up to 24.0 MB of unrelated dirty data, a cost comparable to the kernel's own runtime. With that flush the 1D and 2D kernels measured equally fast at 1080p, although Nsight Compute shows a clear gap. We now flush by reading, report both device time and host latency, verify outputs before timing, and generate every published number from stored results. A fourth pitfall concerns correctness: PyTorch 2.5.1+cu121's TorchDynamo mistraces the `y.extend(m(y[-1]) for m in ...)` pattern in Ultralytics' `C2f`/`SPPF` blocks, so a `torch.compile`d YOLOv8n returned wrong boxes without any error. We rewrite those methods as explicit loops and test bit-exact agreement with eager.

Limitations. One laptop GPU under WSL2, whose clocks and scheduler add variance (we report block spreads); batch 1 only; FP32 models with cuDNN's default TF32 convolutions; no TensorRT or Jetson evaluation; latency only, no accuracy.

7 Conclusion

On an RTX 4050 Laptop GPU, fused Triton kernels for batch-1 preprocessing and YOLO candidate filtering are much faster than PyTorch eager on the GPU, match or beat TorchInductor for preprocessing (up to 1.56×), and cut the filter's host latency 1.62×. Yet they do not measurably change end-to-end latency, which is set by kernel-launch overhead: CUDA graphs reduce it by about half, while the custom kernels' remaining effect stays within run-to-run variation. Pipeline optimization at batch 1 should start by removing launches and synchronizations, and every speedup claim should state which of device time or host latency it measures.

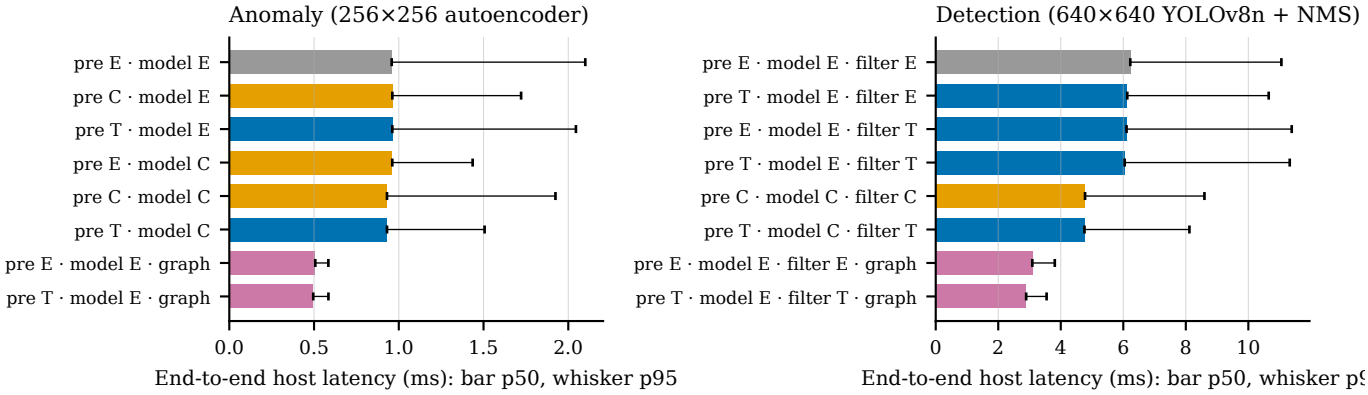


Figure 3: End-to-end host latency per frame (bar: p50, whisker: p95). Labels give the backend of each stage (E: eager, C: TorchInductor, T: Triton); "graph" marks CUDA-graph capture.

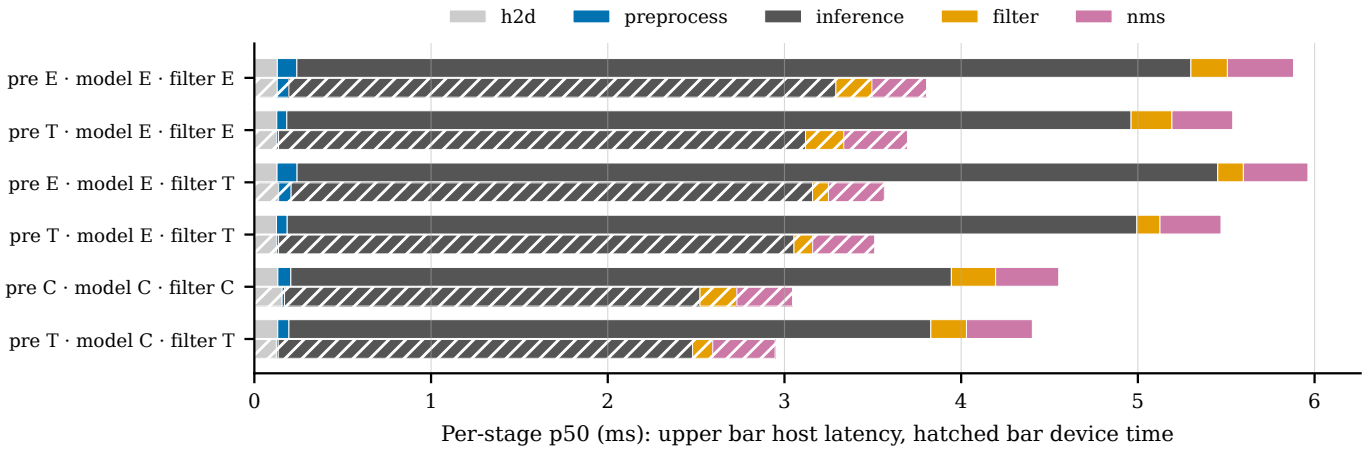


Figure 4: Detection pipeline per-stage p50: host latency (solid) vs device time (hatched). Labels as in Figure 3.

References

[1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[2] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.

[3] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski *et al.*, "PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 929–947.

[4] NVIDIA Corporation, "CUDA C++ programming guide: CUDA graphs," https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2025.

[5] ——, "Nsight Compute profiling guide," https://docs.nvidia.com/nsight-compute/ProfilingGuide/, 2025.

[6] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 519–530.

[7] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, K. Hu, L. Ceze *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.

[9] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS)*, 1967, pp. 483–485.

[10] X. Mei and X. Chu, "Dissecting GPU memory hierarchy through microbenchmarking," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 1, pp. 72–86, 2017.

[11] NVIDIA Corporation, "NVIDIA DALI: Data loading and augmentation library for deep learning applications," https://github.com/NVIDIA/DALI, 2023.

[12] ——, "NVIDIA TensorRT: High-performance deep learning inference sdk," https://developer.nvidia.com/tensorrt, 2024.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8, version 8.0.0," https://github.com/ultralytics/ultralytics, 2023.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[15] D. Merrill and M. Garland, "Single-pass parallel prefix scan with decoupled look-back," NVIDIA, Tech. Rep. NVR-2016-002, 2016.

[16] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD–a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.